

And that's the Angular 2 equivalent of creating a filter, so let's go implement it inside our component.

Using pipes in templates

To use our pipe, we can create a component, add our OrdinalPipe import to the @NgModule inside the declarations Array, and we're good to go.

<http://pastebin.com/bYx99FwV>

Passing arguments to pipes

Passing arguments is pretty much the same as passing any values to a function

```

• import { Pipe, PipeTransform } from '@angular/core';
•
• @Pipe({name: 'ordinal'})
• export class OrdinalPipe implements PipeTransform {
•   // passing another argument
•   transform(value: number, anotherValue: string): string {
•     // do something with `value` and `anotherValue`
•     // and return a new value
•   }
• }

```

Filtering in Component classes with pipes

We can do filtering on an incoming or outgoing dataset in a component using pipes. This can be used to clean up incoming user data or distill data to be sent for display.

First, we need to tell the component that it has a provider:

```

• ...
• import {OrdinalPipe} from './ordinal.pipe';
• @Component({
•   selector: 'my-app',
•   template: `
•     ...
•   `,
•   providers: [OrdinalPipe]
• })
• ...

```